Performance of YUMI

Prajwal Niraula

`YUMI` is a fortran based code written by Nejmeddine Jadaine, which solves for the Schrödinger's equation using Ricatti's method. Using the pre-calculated potential, `YUMI` aims at calculating relevant cross-sections among the interacting molecules, the first of which will be $CO_2 - H_2$. There are essentially two major subroutines that are called `V03` and `ijohnson`, and generally speaking the code spends 4%-15% of time in `V03` and around 85%-95% in `ijohnson`.

Bottle-Neck Identification:

Speed Bottle-Neck

We identify the matrix inversion operations in `ijohnson` are the current speed bottleneck in the `YUMI` code. There are 3 matrix inversions per each step of integration. At energies larger than 1000 cm^{-1}, more than 3 channels of hydrogen are likely to open, these matrices reach the size of 10000×10000 matrices. Our matrix inversion codes are highly optimized routines wrapping `Lapack` algorithm performing block-wise methodology as prescribed in Ingemarsson and Gustafsson [2015] capable of scaling with the number of cores available. Cholesky decomposition is used wherever the matrices are symmetric positive definite Matrix. Further optimization on matrix inversion could further speed-up the performance by a factor up to 2.

Memory Bottle-Neck

`YUMI` allocates and deallocates the memory for the requisite variables, the largest of which is `fl` matrix whose dimensions are given by nlev×nlev×$\lambda_{\max}$. When energy levels are around 1200, the number of levels (nlev) is close to 10000, and there are around 156 values of $\lambda_{\max}$ pre-determined from the potential file. Thus, `fl` matrix is roughly 125 GB of memory. Because of this large memory footprint, only a single case of `YUMI` can be successfully in one node. When `LLMapReduce` is used to submit multiple jobs, only one of the submitted jobs ran, while others crashed.

FLOPS Calculation:

This step of integration (`nPas`) is determined for all different energies and is given by the following formula:

$$\text{nPas} = \frac{\pi}{10\sqrt{2\mu \text{E}}} \tag{1}$$

They scale roughly with the number of square root of energy. For reference, at an energy of 300 the step size is roughly 450 whereas at an energy of 1000 it is 820. Thus, going to higher energies will take longer time due to decrease in the integration step as well as the increase in the size of the matrices.

Each of the matrix inversions using the currently implemented blockwise operation takes roughly takes following number of operations [Ingemarsson and Gustafsson, 2015]:

$$\text{Addition:} \quad \frac{1}{2}N^3 - N \tag{2}$$

$$\text{Multiplication:} \quad \frac{1}{2}\left(N^3 + N^2 - N\right) \tag{3}$$

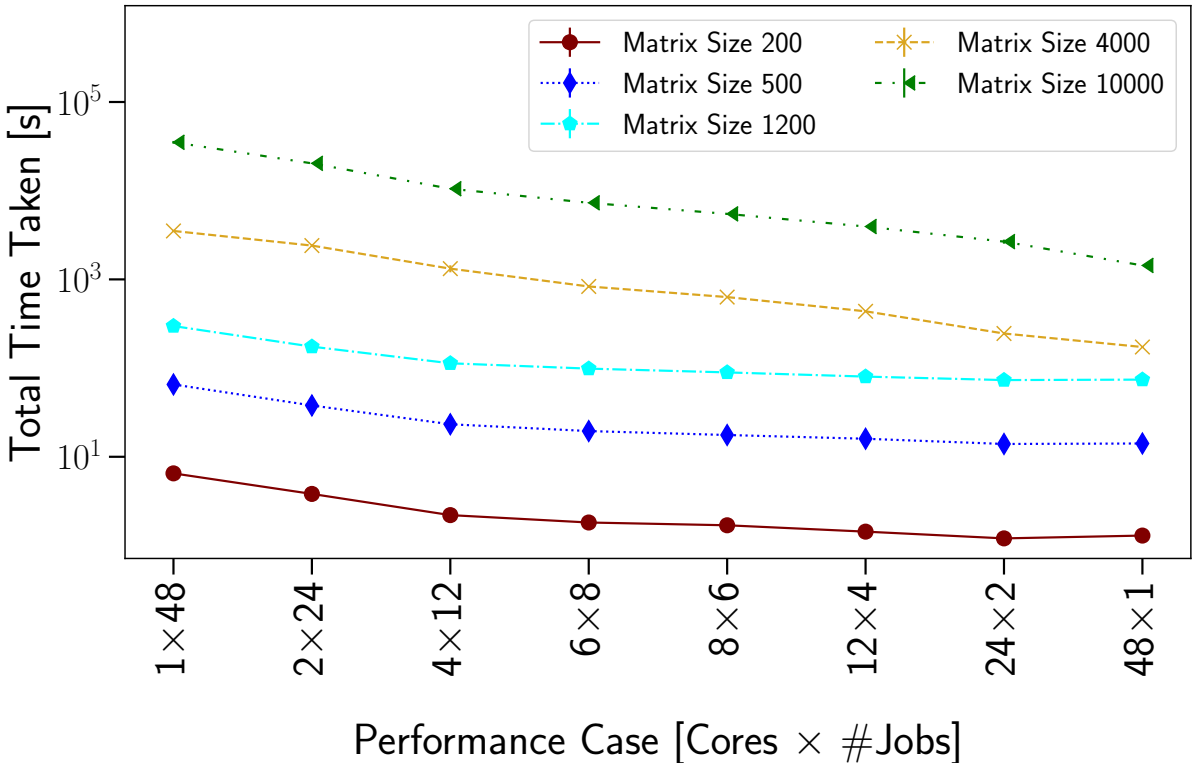

Figure 1: Total computational time taken for different matrix sizes.

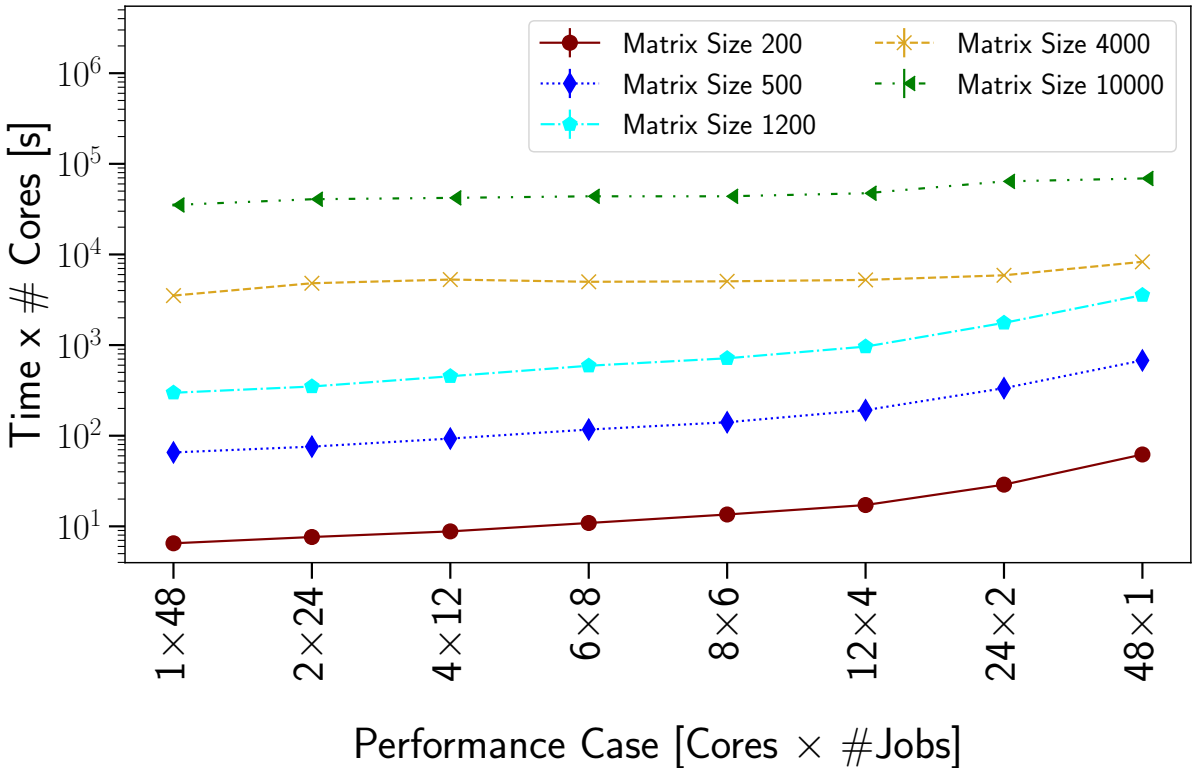

Figure 2: Performances of different settings when normalized by the number of cores for different matrix sizes.

This in-turn can be used to calculate the operation per cycle for the `YUMI`, which is tabulated in the Table .

Matrix Size	Total Time	N. of Op. [$\times 10^9$]	Op. per Time [GigaFlops]	Parallel Part
200	1.202	5.413	1.202	0.848 ± 0.020
500	13.943	84.459	6.06	0.830 ± 0.017
1200	73.246	1166.885	15.93	0.801 ± 0.020
4000	245.46	43205.396	176.01	0.888 ± 0.114
10000	2667.00	675033.74	253.10	0.943 ± 0.043

Expected Speed Up:

If P is the time spent in the parallelizable part of the code, the expected speed-up is given by Amdahl's law as follow:

$$S = \frac{1}{(1-P) + \frac{P}{N}} \tag{4}$$

Since the value of speed-up (S) and number of cores (N) are known, we can

$$P = \frac{S-1}{S \cdot \left(1 - \frac{1}{N}\right)} \tag{5}$$

Based on the time we find, it appears roughly 85% of the code is parallelizable (for smaller matrices, see Table for different matrix size). Thus, the maximum speed-up ($N \to \infty$) we should be ever to get will be around $\sim$6.7 compared to single thread operation.However, the value of P scales with the matrix size, and for larger matrices (10000×10000), we see a speed-up factor of 24, when using 48 cores.

GPU Based Code:

As shown in the figure 2, our code is very well optimized for CPU. Given the required working memory of greater than 100 GB is required, it is surmised that using GPU is unlikely to provide substantial improvements. However, a combination of CPU-GPU code depending on the type of calculation, might forge a path for shorter calculation time.

Future Avenues:

We are exploring ways to improve computational performance of `YUMI`. Barring better algorithms, following are some of the potential avenues:

1. We have also implemented on the run the convergence test, so that we can stop the calculation once the scattering matrix has essentially converged with in 1%. Understanding the convergence of our scattering matrices can help to improve substantial time.
2. Another way to gain speed is by trading off some precision. Currently all of the matrices are calculated at double precision level, perhaps some of this can be traded off.
3. At times, matrices can be sparsely populated, which can open-up newer avenues for the matrix inversion to further optimize the speed.

References

Carl Ingemarsson and Oscar Gustafsson. On fixed-point implementation of symmetric matrix inversion. In *2015 European Conference on Circuit Theory and Design (ECCTD)*, pages 1–4, 2015. doi: 10.1109/ECCTD.2015.7300068.